

UNIVERSIDAD NACIONAL DEL ALTIPLANO - PUNO
FACULTAD DE INGENIERIA MECANICA ELECTRICA, ELECTRONICA Y
SISTEMAS
ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS



INFORME TECNICO

**FrostPuno: sistema de prediccion de heladas y apoyo a la
produccion de chuno en el altiplano de Puno mediante aprendizaje
no supervisado**

Aplicacion desplegada en produccion con procesos automatizados de mantenimiento e
integracion continua

Curso	Aprendizaje de Maquina
Proyecto	Segunda Unidad
Docente	Ing. FERNANDEZ CHAMBI MAYENKA
Integrantes	Mamani Mendoza, Joseph Elvis Lipe Machaca, Juan Artemio Ticona Erquinigo, Jhoel Yovani Tapara Ccahuana, Paul Renmis
Lugar y fecha	Puno, Peru - 2026

Enlaces del producto y del entregable

El sistema descrito en este informe se encuentra operativo. Los siguientes enlaces permiten verificar cada componente de forma directa.

Tabla 1. Enlaces de verificación del producto entregado.

Recurso	Enlace
Video de la exposicion	https://drive.google.com/drive/folders/1nxD0mXwB2P-LHh588sDdEh17SuBmWok7?usp=sharing
Repositorio de codigo fuente (GitHub)	https://github.com/JosephElvisMaman1/frost-puno
Aplicacion web (Flutter en Vercel)	https://frost-puno.vercel.app
API de produccion (FastAPI en Render)	https://frost-puno.onrender.com
Documentacion interactiva de la API	https://frost-puno.onrender.com/docs
Aplicacion Android (APK release)	https://github.com/JosephElvisMaman1/frost-puno (build: app-release.apk, 47.3 MB)

El archivo APK se genera desde el repositorio con el siguiente comando y queda disponible en la ruta indicada:

```
flutter build apk --release --dart-define=API_BASE_URL=https://frost-puno.onrender.com
Salida: app_flutter/build/app/outputs/flutter-apk/app-release.apk
```



Figura 1. Verificación de integridad del APK Android mediante Play Protect: sin virus detectados y aplicación legítima.

1. Introduccion

FrostPuno es una aplicacion con elementos inteligentes orientada a la prediccion del riesgo de heladas en el altiplano de la region Puno. Por encima de los 3 800 metros sobre el nivel del mar, las heladas afectan los cultivos y provocan mortalidad neonatal en crias de alpaca y ovino. Esas mismas heladas constituyen el insumo del chuno, la papa deshidratada mediante congelamiento nocturno y secado diurno que forma parte de la seguridad alimentaria regional.

La logica de negocio del sistema no se resuelve con reglas fijas: se operativiza a traves de las predicciones de un modelo de aprendizaje no supervisado, concretamente un modelo de agrupamiento K-Means que descubre los regimenes climaticos de los distritos y los asocia a niveles de riesgo.

Este documento esta dirigido a un equipo de tecnologias de la informacion que reciba el sistema para su mantenimiento e integracion. Por ello describe las herramientas necesarias, la organizacion del codigo, las consideraciones de despliegue inicial y los flujos automatizados que mantienen operativo tanto el software como el modelo.

1.1 Objetivo del sistema

- Emitir una alerta diaria de riesgo de helada con umbral configurable por el usuario.
- Diferenciar el riesgo para ganado del riesgo para cultivos.
- Identificar ventanas optimas para la elaboracion de chuno durante la temporada.
- Agrupar los distritos de Puno por nivel de riesgo mediante aprendizaje no supervisado.
- Detectar eventos climaticos inusuales respecto de la climatologia reciente.

1.2 Trazabilidad academica

El proyecto se inicio con un clasificador supervisado basado en RandomForest evaluado con la metrica f1 macro. Conforme al enunciado de la segunda unidad, se migro a aprendizaje no supervisado. Los scripts del modelo anterior se conservan en el repositorio como referencia documentada, fuera de la ruta productiva, de modo que la evolucion del proyecto sea auditable.

2. Herramientas, plataformas y aplicaciones necesarias

Esta seccion detalla el software y los servicios requeridos para desplegar, mantener e integrar el sistema. Todas las plataformas empleadas cuentan con un plan gratuito suficiente para el alcance academico del proyecto.

2.1 Entorno de desarrollo

Tabla 2. Herramientas de desarrollo y sus versiones.

Herramienta	Version	Funcion en el proyecto
Python	3.11	Pipeline de aprendizaje automatico y servicio backend
scikit-learn	1.5 o superior	Implementacion de K-Means, escalado y metricas
pandas	2.2 o superior	Construccion y validacion del conjunto de datos
joblib	1.4 o superior	Serializacion del modelo entrenado
FastAPI	0.115 o superior	Servicio web que expone el modelo
Uvicorn	0.32 o superior	Servidor ASGI de ejecucion del backend
Pydantic	2.9 o superior	Validacion de esquemas de entrada y salida
httpx	0.27 o superior	Cliente HTTP hacia el proveedor de clima
Flutter	3.x	Cliente web y aplicacion Android
Git	2.x	Control de versiones

2.2 Plataformas de despliegue y operacion

Tabla 3. Plataformas de despliegue, mantenimiento e integracion continua.

Plataforma	Plan	Responsabilidad
Render	Free	Alojamiento del backend FastAPI y del modelo en memoria
Vercel	Hobby	Alojamiento del cliente Flutter Web mediante red de distribucion
Supabase	Free	Base de datos PostgreSQL gestionada para el historial

GitHub	Publico	Repositorio de codigo y ejecucion de la integracion continua
GitHub Actions	Incluido	Automatizacion de pruebas, entrenamiento y control de calidad
Open-Meteo	API abierta	Fuente climatica historica, de pronostico y de archivo

2.3 Dependencias de la aplicacion movil

- geolocator: obtencion de coordenadas por sistema de posicionamiento global.
- permission_handler: gestion de permisos de ubicacion y notificaciones en Android.
- fl_chart: graficos de temperatura y humedad en la pantalla de clima historico.
- shared_preferences: persistencia local de umbral de alerta, perfil e idioma.
- flutter_local_notifications: notificaciones nativas de alerta de helada en Android.
- flutter_localizations: soporte de los idiomas espanol e ingles en la interfaz.

2.4 Requisitos minimos para el mantenimiento

Un equipo que reciba el sistema requiere unicamente: Python 3.11, Flutter, Git, una cuenta de GitHub con permisos sobre el repositorio y acceso a los paneles de Render, Vercel y Supabase. No se necesita infraestructura propia ni licencias de pago.

3. Organizacion del codigo fuente

El repositorio se organiza en tres subsistemas independientes mas los datos compartidos y la definicion de la automatizacion. Cada subsistema puede evolucionar por separado porque la comunicacion entre ellos ocurre a traves de contratos explicitos.

3.1 Estructura de directorios

ml_pipeline/	Pipeline de aprendizaje automatico
config.py	Fuente unica de rutas, características y constantes
data_ingestion/	Ingesta de clima y ubicaciones
features/	Construccion del conjunto de datos
clustering/	Entrenamiento K-Means y reentrenamiento adaptativo
registry/	Modelo serializado, metadatos y control de calidad
evaluation/	Perfiles de agrupamiento y reportes
tests/	Pruebas de mantenimiento e integracion continua
backend_fastapi/	Servicio web que expone el modelo
app/api/routes/	Capa HTTP
app/services/	Logica de negocio
app/repositories/	Persistencia
app/schemas/	Contratos de datos con Pydantic
app/core/	Configuracion y excepciones
tests/	Pruebas del servicio
app_flutter/	Cliente web y Android
lib/core/	Cliente HTTP, tema, preferencias e idiomas
lib/features/	Una carpeta por funcionalidad
lib/shared/widgets/	Componentes reutilizables
data/	Conjuntos de datos versionados
.github/workflows/	Cinco flujos de integracion continua
docs/	Documentacion tecnica y evidencias

3.2 Principios de organizacion aplicados

- **Configuracion centralizada.** La configuracion esta centralizada. El archivo ml_pipeline/config.py concentra rutas, listas de características y constantes; el backend hace lo propio en app/core/config.py y el cliente recibe su configuracion por parametro de compilacion. No existen rutas escritas dentro de la logica.
- **Separacion por capas.** El backend separa la capa HTTP de la logica y de la persistencia. Cada ruta delega en un servicio y este en un repositorio, de modo que la fuente de datos puede cambiar sin modificar los controladores.
- **Organizacion por funcionalidad.** El cliente Flutter agrupa el codigo por funcionalidad y no por tipo de archivo. Cada carpeta bajo lib/features contiene sus pantallas, modelos y servicios.
- **Contrato explicito entre modelo y servicio.** La lista de características declarada en los metadatos del modelo define el contrato entre el pipeline y el servicio de inferencia. El backend construye el vector de entrada siguiendo ese orden, por lo que un reentrenamiento no obliga a modificar el codigo del servidor.

- **Gestion de errores en el borde.** Los errores internos se transforman en respuestas estables de la interfaz de programacion. No se exponen detalles de implementacion del modelo al cliente.

3.3 Convenciones de trabajo

El desarrollo se realiza en ramas y la integracion a la rama principal se efectua mediante solicitudes de incorporacion, lo que permite que los flujos de verificacion se ejecuten antes de que un cambio llegue a produccion. Los conjuntos de datos estan versionados y su modificacion exige aprobar el flujo de validacion de datos.

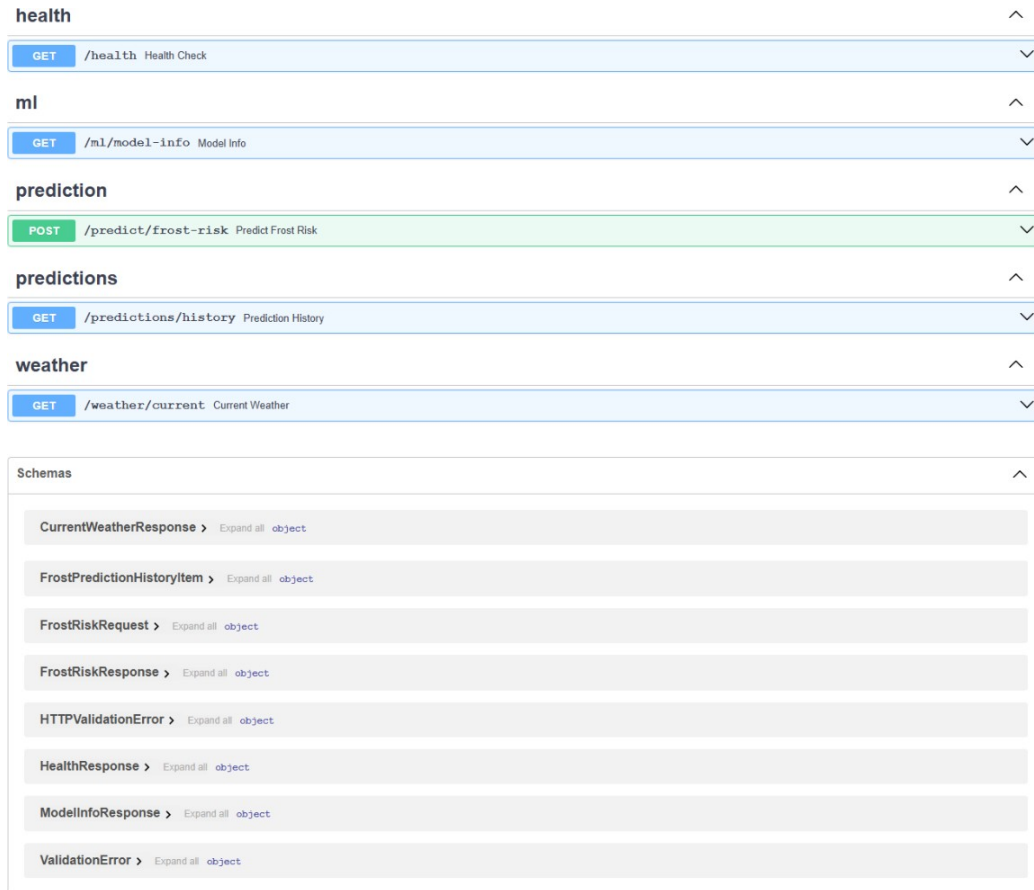


Figura 2. Documentacion interactiva de la interfaz de programacion generada automaticamente por FastAPI y disponible en produccion.

4. Consideraciones de despliegue inicial

El sistema se distribuye en cuatro componentes alojados en plataformas distintas. Esta seccion documenta la configuracion necesaria y las precauciones detectadas durante la puesta en produccion.

4.1 Arquitectura de despliegue

Tabla 4. Distribucion de los componentes en produccion.

Componente	Plataforma	Direccion
Cliente Flutter Web	Vercel	https://frost-puno.vercel.app
Servicio FastAPI	Render	https://frost-puno.onrender.com
Base de datos PostgreSQL	Supabase	Instancia gestionada del proyecto
Aplicacion Android	Distribucion directa	app-release.apk

4.2 Configuracion del servicio backend

El despliegue del backend esta declarado como infraestructura como codigo en el archivo render.yaml, de modo que la configuracion es reproducible y queda versionada junto al codigo.

```
runtime: python 3.11
buildCommand: pip install -r backend_fastapi/requirements.txt
startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT --app-dir
                backend_fastapi
healthCheckPath: /health
autoDeploy: true
```

Las variables de entorno requeridas por el servicio son las siguientes:

Tabla 5. Variables de entorno del servicio backend.

Variable	Proposito
MODEL_PATH	Ruta del modelo de agrupamiento serializado
MODEL_METADATA_PATH	Ruta de los metadatos con características y métricas
DISTRICT_CLUSTERS_PATH	Ruta de la agrupacion de distritos por nivel de riesgo
CORS_ORIGINS	Origenes autorizados para el consumo desde el cliente
ENABLE_SUPABASE	Activa o desactiva la persistencia en base de datos
SUPABASE_URL	Direccion del proyecto de base de datos
SUPABASE_SERVICE_ROLE_KEY	Credencial de servicio, declarada como no

	sincronizable
--	---------------

4.3 Consideracion critica detectada en produccion

Durante la migracion del modelo supervisado al modelo de agrupamiento se produjo una situacion que conviene documentar para el equipo que reciba el sistema. Tras incorporar los cambios a la rama principal, la plataforma redespago el servicio correctamente, pero la interfaz de programacion continuo respondiendo con el modelo anterior.

La causa fue que las variables de entorno definidas manualmente en el panel de administracion tienen prioridad sobre las declaradas en el archivo de configuracion. Las rutas del modelo anterior permanecian fijadas en el panel, por lo que el archivo versionado no surtia efecto. La verificacion recomendada despues de cada cambio de modelo consiste en consultar el punto de acceso de informacion del modelo y confirmar que el nombre y la version corresponden al artefacto esperado.

4.4 Configuracion del cliente

La direccion del servicio no esta escrita en el codigo del cliente. Se inyecta en tiempo de compilacion, de manera que el mismo codigo fuente sirve para el entorno local, para la version web y para la aplicacion Android.

```
flutter build web --release --dart-define=API_BASE_URL=https://frost-
puno.onrender.com
flutter build apk --release --dart-define=API_BASE_URL=https://frost-
puno.onrender.com
flutter run -d chrome --dart-define=API_BASE_URL=http://127.0.0.1:8000
```

4.5 Secuencia de puesta en marcha

1. Entrenar el modelo y publicar los artefactos en el registro de modelos.
2. Desplegar el servicio backend y verificar los puntos de acceso de salud e informacion del modelo.
3. Desplegar el cliente web indicando la direccion del servicio.
4. Compilar la aplicacion Android apuntando al mismo servicio.
5. Activar la persistencia en base de datos de forma opcional. Con la persistencia desactivada el sistema opera con un repositorio en memoria, que es la configuracion utilizada por las pruebas automatizadas.

4.6 Consideraciones de seguridad

- La credencial de servicio de la base de datos existe unicamente en el panel del backend y esta marcada como no sincronizable, por lo que nunca se incorpora al repositorio.
- El cliente recibe exclusivamente la direccion publica del servicio y en ningun caso credenciales de base de datos.
- El servicio valida el rango de coordenadas correspondiente a la region de Puno antes de procesar cualquier consulta.

4.7 Limitaciones del entorno gratuito

- El plan gratuito del servicio backend suspende la instancia tras un periodo de inactividad, por lo que la primera petición puede demorar mientras el servicio se reactiva.
- La aplicación Android no está firmada para distribución en tiendas, se entrega como archivo de instalación directa.

```
'''text
app_flutter/web/icons/
app_flutter/web/favicon.png
'''

## Ejecutar en Android

'''powershell
Set-Location app_flutter
flutter run -d emulator --dart-define-API_BASE_URL=http://10.0.2.2:8080
'''

Para dispositivo físico con backend desplegado:

'''powershell
flutter run -d android --dart-define-API_BASE_URL=https://frost-puno.onrender.com
'''

## Generar APK

'''powershell
Set-Location app_flutter
flutter build apk --release --dart-define-API_BASE_URL=https://frost-puno.onrender.com
'''

Salida esperada:

'''text
app_flutter/build/app/outputs/flutter-apk/app-release.apk
'''

## Generar App Bundle

'''powershell
flutter build appbundle --release --dart-define-API_BASE_URL=https://frost-puno.onrender.com
'''

## Probar GPS

1. Abrir FrostPuno en Android o Flutter Web HTTPS.
2. Ir a 'Consultar riesgo'.
3. Tocar 'Usar mi ubicación'.
4. Aceptar permiso de ubicación.
5. Confirmar que latitud/longitud cambien.
6. Confirmar que el bloque de clima indique proveedor y fallback.
7. Ejecutar predicción.

## PWA

El proyecto incluye 'web/manifest.json', 'web/offline.html' y service worker generado por Flutter durante 'flutter build web'.

'''powershell
flutter build web --release --dart-define-API_BASE_URL=https://frost-puno.onrender.com
'''

## Limitaciones actuales

- No hay modo offline-first complejo; clima y predicciones requieren red.
- El APK no está firmado para Play Store.
- GPS en navegador requiere HTTPS y permiso del usuario.
- SENAMHI está preparado como proveedor prioritario, pero el MVP usa Open-Meteo como fallback cuando no hay API pública estable configurada.
```

Fig. 5. Aplicación Android desde el APK release.

Figura 3. Respuesta del punto de acceso de verificación de salud del servicio desplegado.

5. Flujos de mantenimiento e integracion continua

El sistema incorpora un componente inteligente, por lo que el mantenimiento no se limita al código: también comprende el reentrenamiento del modelo, el registro de versiones, el almacenamiento de características y el control de calidad previo a la promoción de un modelo nuevo. Todos estos procesos están automatizados.

5.1 Flujos de integracion continua

Se implementaron cinco flujos independientes que se ejecutan de forma simultanea en entornos separados ante cada incorporacion de cambios al repositorio.

Tabla 6. Flujos de integracion continua implementados.

Flujo	Disparo	Funcion
ml-training.yml	Manual y programado	Ingesta de clima, construccion de características, entrenamiento y publicacion de artefactos
model-quality-gate.yml	Cada cambio	Verifica la calidad del modelo y ejecuta las pruebas de mantenimiento
backend-tests.yml	Cada cambio	Ejecuta las doce pruebas del servicio sin base de datos externa
data-validation.yml	Cada cambio	Valida la estructura de los conjuntos de datos y la agrupacion de distritos
flutter-build.yml	Cada cambio	Analiza, prueba y compila la version web del cliente

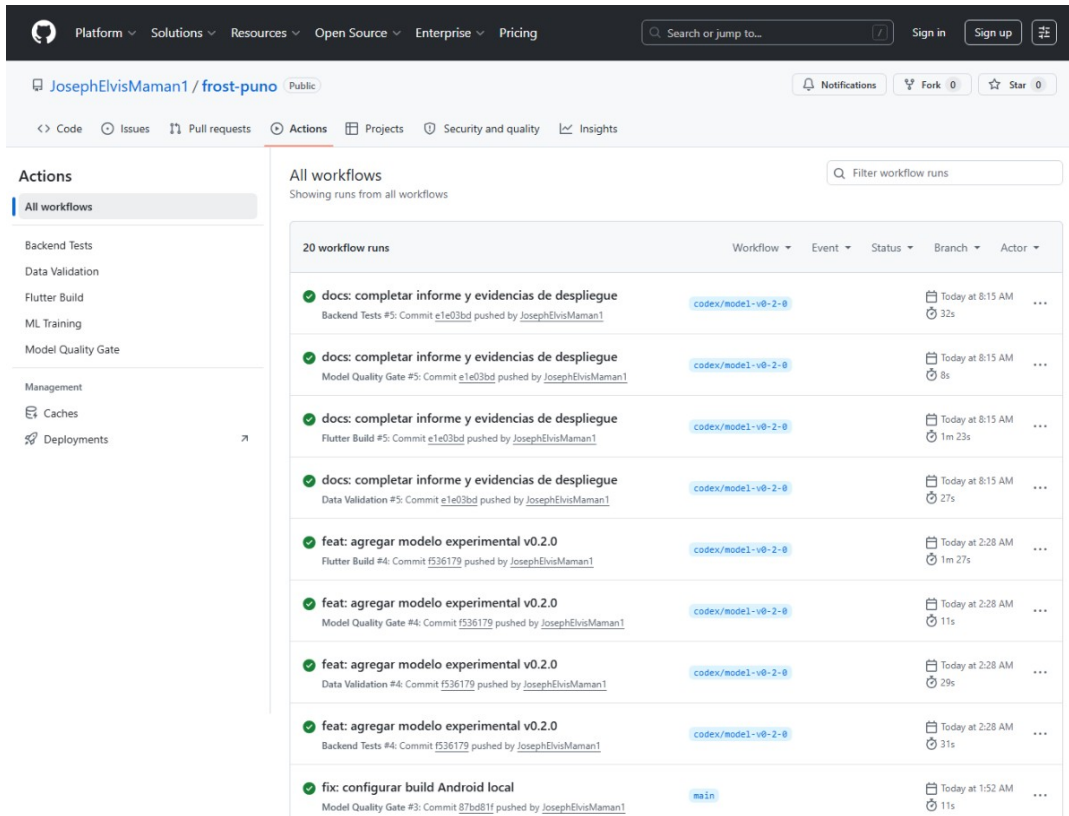


Figura 4. Ejecuciones de los flujos de integracion continua en estado satisfactorio.

5.2 Mantenimiento del modelo

El modelo se reentrena de forma automatica sin intervencion humana. El flujo de entrenamiento cuenta con una programacion semanal mediante expresion de tiempo, de manera que el sistema incorpora datos climaticos recientes de forma periodica.

Reentrenamiento adaptativo.

El procedimiento de reentrenamiento no se limita a repetir el entrenamiento con los mismos parametros. Si la calidad de agrupamiento obtenida no alcanza el objetivo definido, el procedimiento amplia la ventana de datos historicos, de catorce a veintinueve y luego a cuarenta y cuatro dias, y repite el entrenamiento. Cada ejecucion queda registrada en un historial que incluye la fecha, la cantidad de dias de datos empleados, el tamano del conjunto resultante, la metrica obtenida y los hiperparametros seleccionados. Ese historial constituye la evidencia de que el modelo mejora conforme dispone de mas informacion.

Registro de modelos.

Cada entrenamiento serializa el modelo junto con un archivo de metadatos que contiene la lista de caracteristicas, las metricas obtenidas, los hiperparametros explorados y los elegidos, el tamano del conjunto de datos, la version y las limitaciones declaradas. El

servicio backend carga exactamente ese artefacto, lo que garantiza la correspondencia entre lo entrenado y lo servido.

Almacenamiento de características.

El conjunto de características se encuentra versionado dentro del repositorio y se valida en cada integracion. Cumple la funcion de almacen de características del proyecto, de modo que cualquier reentrenamiento parte de una base reproducible.

5.3 Control de calidad como barrera

El reentrenamiento automatico presenta un riesgo: un modelo nuevo podria resultar inferior al que se encuentra en produccion. Para evitarlo se implemento un control de calidad que verifica la metrica de agrupamiento antes de permitir la promocion. Si el valor obtenido se encuentra por debajo del umbral establecido de 0.35, el proceso finaliza con codigo de error y la integracion se detiene.

```
python -m ml_pipeline.registry.check_cluster_quality --min-silhouette 0.35
Resultado: QUALITY GATE PASSED (silhouette 0.419)
```

5.4 Pruebas de funcionamiento del mantenimiento

La existencia de los flujos no demuestra por si sola que el mantenimiento opere de forma correcta. Por ese motivo se implementaron seis pruebas automatizadas que se ejecutan dentro de la integracion continua y verifican el comportamiento del ciclo completo.

Tabla 7. Pruebas automatizadas del mantenimiento y la integracion continua.

Prueba	Comportamiento verificado
Artefactos del registro	El reentrenamiento genera el modelo, los metadatos y la agrupacion de distritos
Carga e inferencia	El artefacto serializado se carga y produce predicciones, manteniendo el contrato con el servicio
Cobertura territorial	Los trece distritos quedan asignados a un nivel de riesgo valido
Hiperparametros documentados	Los metadatos registran la busqueda realizada y la configuracion elegida corresponde a la mejor evaluada
Aprobacion del control de calidad	El control aprueba el modelo vigente en produccion
Bloqueo de modelo degradado	El control rechaza un modelo con metrica insuficiente y finaliza con codigo de error

La ultima prueba resulta particularmente relevante porque verifica el funcionamiento de la propia barrera de seguridad: construye deliberadamente un conjunto de metadatos con una metrica insuficiente y comprueba que el control lo rechaza.

```
python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v
Resultado: 6 passed
```

5.5 Evidencias de funcionamiento

Tabla 8. Resumen de las verificaciones realizadas sobre el sistema.

Verificacion	Resultado
Pruebas del servicio backend	12 pruebas satisfactorias
Pruebas de mantenimiento e integracion continua	6 pruebas satisfactorias
Control de calidad del modelo	Aprobado, metrica 0.419 frente a un minimo de 0.35
Analisis estatico y pruebas del cliente	Sin observaciones
Compilacion de la version web	Satisfactoria
Compilacion de la aplicacion Android	Satisfactoria, 47.3 MB
Modelo servido en produccion	KMeans, version v1.0.0-clustering

6. Componente inteligente del sistema

Aunque la evaluacion de este informe se centra en el despliegue y la automatizacion, se incluye una descripcion del modelo para que el equipo de mantenimiento comprenda que se esta operando.

6.1 Conjunto de datos y características

La fuente climatica es Open-Meteo, de la que se obtienen series horarias para trece distritos de la region, lo que produce un conjunto de 4 368 registros. La informacion territorial procede de una semilla compatible con el Instituto Nacional de Estadistica e Informatica. El Servicio Nacional de Meteorologia e Hidrologia se declara como fuente oficial prioritaria en el sistema, si bien no expone una interfaz publica estable, por lo que Open-Meteo constituye la fuente operativa.

El modelo utiliza cuatro características: altitud estimada, temperatura, punto de rocío y humedad relativa. La seleccion se realizo mediante un experimento de subconjuntos que evidencio que la precipitacion, la velocidad del viento, la nubosidad y la temperatura aparente introducian ruido y reducian la separacion entre grupos. Al retirarlas, la metrica de agrupamiento aumento de 0.286 a 0.419.

6.2 Entrenamiento e hiperparametros

El algoritmo es K-Means, incorporado en una secuencia de procesamiento que aplica previamente un escalado estandar, necesario porque las variables presentan magnitudes muy distintas. La seleccion de hiperparametros se realiza mediante una busqueda exhaustiva de veinticuatro combinaciones que maximiza la metrica de agrupamiento.

Tabla 9. Busqueda de hiperparametros y configuracion seleccionada.

Hiperparametro	Valores evaluados	Valor seleccionado
Numero de grupos	3, 4, 5, 6, 7, 8	3
Metodo de inicializacion	k-means++, aleatorio	aleatorio
Numero de inicializaciones	10, 25	10
Escalado	Estandarizacion	Aplicado
Semilla de aleatoriedad	Fija	42

6.3 Evaluacion

Al tratarse de aprendizaje no supervisado no se dispone de etiquetas de referencia, por lo que no procede el uso de exactitud ni de f1. La evaluacion mide la calidad de la estructura descubierta mediante el coeficiente de silueta, que alcanza un valor de 0.419, y el indice de Davies-Bouldin, que alcanza un valor de 0.813.

El resultado admite interpretacion geografica. El distrito de Macusani, situado a 4 315 metros, queda clasificado en riesgo alto, mientras que Sandia, ubicado en valle a 2 170

metros, queda en riesgo bajo. Esta correspondencia con la geografía regional no fue programada: emerge del agrupamiento.



Figura 5. Pantalla de zonas de riesgo con las metricas del modelo en produccion y la clasificacion de distritos por nivel.

7. Funcionamiento de la aplicación

La aplicación se encuentra disponible como versión web y como aplicación Android. Incorpora selección de idioma entre español e inglés, de modo que la totalidad de la interfaz, incluidas las alertas y los niveles de riesgo, puede presentarse en inglés.

7.1 Funcionalidades principales

- Alerta diaria de helada con umbral de temperatura configurable por el usuario.
- Aviso diferenciado de riesgo para el ganado, orientado a la protección de crías.
- Detección de eventos inusuales por comparación con la climatología reciente.
- Agrupación de distritos por nivel de riesgo obtenida del modelo.
- Módulo estacional de chuno con evaluación de ventanas de congelamiento y secado.
- Histórico climático de los últimos treinta días con representación gráfica.
- Perfil de usuario que prioriza las alertas mostradas según la actividad.



Figura 15
Pantalla "Inicio" de FrostPuno en tema claro, modo demo: mismo riesgo actual de helada (-4.2 C, humedad 42%, viento 12 km/h) que la Figura 14, mostrando el soporte de temas claro/oscuro del cliente Flutter.

Figura 6. Pantalla principal con la alerta del día y el riesgo actual.



Figura 7. Modulo estacional de chuno con la evaluacion del pronostico diario.

8. Limitaciones

Se declaran de forma explicita las limitaciones del sistema, dado que su conocimiento resulta necesario para cualquier equipo que asuma el mantenimiento.

- Los niveles de riesgo se derivan del perfil termico de cada grupo y no de un registro oficial de heladas observadas, dado que no existe un conjunto etiquetado disponible para estos distritos.
- El Servicio Nacional de Meteorologia e Hidrologia no expone una interfaz publica estable, por lo que la fuente operativa efectiva es Open-Meteo.
- La semilla territorial constituye una version reducida y debe sustituirse por la exportacion oficial completa para un uso productivo.
- Los umbrales aplicados al chuno y al ganado corresponden a un diseno documentado con fuentes, sin constituir cifras oficiales.
- La deteccion de eventos inusuales emplea climatologia reciente de treinta dias y no una serie multianual.
- El plan gratuito de la plataforma de alojamiento introduce una demora en la primera peticion tras un periodo de inactividad.

Ninguna de estas limitaciones afecta la validez de la arquitectura de despliegue ni de los procesos de mantenimiento e integracion continua descritos en este informe.

9. Conclusiones

El sistema cumple con los objetivos planteados para la segunda unidad. Se desarrollo una aplicacion con elementos inteligentes basada en aprendizaje no supervisado, se desplego en produccion y se automatizaron sus procesos de mantenimiento e integracion continua.

6. El componente inteligente corresponde a un modelo de agrupamiento K-Means con hiperparametros optimizados mediante busqueda exhaustiva, evaluado con metricas propias del aprendizaje no supervisado.
7. La aplicacion se encuentra operativa y accesible como version web, como interfaz de programacion y como aplicacion Android.
8. El mantenimiento del modelo esta automatizado mediante reentrenamiento programado con ampliacion adaptativa de la ventana de datos, registro de versiones y almacenamiento de caracteristicas.
9. La integracion continua comprende cinco flujos y un control de calidad que impide la promocion de modelos degradados.
10. El funcionamiento del mantenimiento se verifica mediante pruebas automatizadas, una de las cuales comprueba que el control de calidad rechaza efectivamente un modelo insuficiente.

10. Video de la exposicion

La exposicion tecnica del producto se encuentra grabada y disponible en el siguiente enlace, con una duracion que se ajusta al maximo de doce minutos establecido.

Recurso	Enlace
Video de la exposicion	https://drive.google.com/drive/folders/1nxD0mXwB2P-LHh588sDdEh17SuBmWok7?usp=sharing

La grabacion comprende la presentacion del problema y la arquitectura, la descripcion del conjunto de datos, la explicacion del entrenamiento del modelo y de los hiperparametros optimizados, la demostracion del funcionamiento de la aplicacion en ingles, el despliegue en produccion y los flujos de mantenimiento e integracion continua con sus pruebas de funcionamiento.

Anexo. Comandos de verificacion

Los siguientes comandos permiten reproducir las verificaciones descritas en este informe a partir del repositorio.

```
# Entrenamiento del modelo
python -m ml_pipeline.clustering.train_clusters

# Control de calidad del modelo
python -m ml_pipeline.registry.check_cluster_quality --min-silhouette 0.35

# Pruebas de mantenimiento e integracion continua
python -m pytest ml_pipeline/tests/test_maintenance_ci.py -v

# Reentrenamiento adaptativo
python -m ml_pipeline.clustering.adaptive_retrain --target 0.45 --max-iters 3

# Pruebas del servicio backend
pytest backend_fastapi/tests -q

# Verificacion del cliente
flutter analyze && flutter test && flutter build web --release
```